

3D Reconstruction: Multi Robot Coordination to Reconstruct a 3D Model Object

Ruben Malacarne 239934^a, Emmanuele V. Coppola 247540^a

^a*University of Trento, Via Sommarive 9, 38123, Trento, Italy*

Abstract

Objective: The project proposes the development of a distributed system for the three-dimensional reconstruction of objects using mobile robots equipped with depth cameras. The system allows for the accurate acquisition of point clouds without use external markers, thanks to computer vision technics and cooperation between robots. The approach combines distributed robotics logic and artificial intelligence to achieve more efficient, scalable, and autonomous approach, with potential applications in advanced manufacturing, industrial inspection, cultural heritage preservation, and digital twin generation for simulation and optimization.

Video about project: [Tutorial to install || Simulation Example](#)

Github: [Click here](#)

1. State of the Art

A wide range of methodologies has been proposed in the literature to address the problem of three-dimensional object search, acquisition and reconstruction, particularly in complex and dynamic environments. In the context of active 3D reconstruction, Next-Best-View (NBV) strategies and multi-robot motion planning algorithms are widely employed to optimize information gain. These methods aim to maximize scene observability and reconstruction completeness while minimizing the number of required viewpoints, traversal time, and sensing latency, visibility, and collision constraints.

Multi-robot architectures can be classified into centralized, decentralized, or hybrid approaches. In fully decentralized systems, each robot makes decisions locally, with high requirements in terms of communication and coordination to ensure global consistency. In centralized systems, on the other hand, a single control unit gathers information from all robots, plans the tasks, and assigns actions, but this introduces potential block or single points of failure. With regard to hybrid architectures, exploration and search are distributed among the robots, while the reconstruction and global optimization phase is delegated to a central node, particularly in high-fidelity 3D reconstruction scenarios.[1].

Our project is inspired by this two-phase hybrid architecture. In the first, decentralized phase, each robot autonomously searches for the target object and performs its detection using on-board sensors and vision-based algorithms. Once the object is detected, the robots coordinate to arrange themselves in suitable viewing configurations around it, maximizing coverage. In the second phase, which is centralized and performed offline, a processing system collects the acquired depth maps and point clouds and executes the three-dimensional reconstruction

algorithm to obtain the final 3D model.

2. Introduction

Three-dimensional reconstruction represents a key element for the generation of three-dimensional models and objects from a physical target, enabling the faithful transfer of complex geometries into the digital domain. Both in the industrial field and in the cultural heritage sector, this technology can help generate educational models, documentation support tools, and detailed analyzes of objects with high accuracy. In many cases, the availability of a 3D model also enables efficient and repeatable modifications, simulations, and further processing.

The project presented here exposes a 3D reconstruction framework based on a fleet of autonomous ground robots operating according to the distributed systems paradigm. Each robot is equipped with a front-facing depth camera, which enables it to search for and detect the target object within the environment, as well as to acquire depth information. For accurate localization and precise positioning in space, sensors such as LiDAR and IMU are employed, ensuring an accurate estimation of the robot pose and a consistent reference within the environment. The core idea of the project is to allow the robots to scan the object in a coordinated manner by autonomously selecting the most informative viewpoints, thereby reducing the number of required iterations while maintaining a high level of accuracy in the final reconstruction.

This report is organized into three main chapters. The first chapter 3, describes the implemented communication system and the type of robots employed. In the following chapter, 4, the proposed solution to address the problem is analyzed in detail by presenting the system architecture and decomposing the project into its fundamental components. Finally, the last section, 5, reports the obtained

results and the corresponding conclusions, discussing the limitations of the proposed approach and possible future developments.

3. Adopted Model

3.1. Communication System

The communication infrastructure between the robots and the coordination server is based on a LAN connection. All coordination and robot status information is mediated by the ROS2 middle ware through topics and services, as described in the network map shown in Fig. 8. Given the need to simulate both the robots, their control logic, sensors, and the entire infrastructure, an exception has been made for the transmission of point clouds to the coordination server. Specifically, the server receives the point clouds in the form of *Polygon File Format* (PLY) files, which are directly uploaded to a working directory in order to reduce the computational load on the machine running the simulation. Outside the simulation environment, this point cloud communication could also be performed by means of dedicated ROS2 messages.

3.2. System Model

For the simulation we have used the Gazebo, which allows the integration of cameras, LiDAR sensors, and encoders, providing a framework that closely resembles real-world conditions. The overall system is composed of multiple robots and a central server. The robots are autonomous unicycle-type platforms, named *Shelfino*, capable of navigating independently within the environment. They are equipped with a pair of differential-drive wheels. The sensing suite includes a two-dimensional LiDAR capable of detecting points on the horizontal plane around the robot by exploiting the Time-of-Flight (ToF) principle, and an RGB-D camera with a resolution of 640×480 pixels, a field of view (FoV) of 63° , and a maximum sensing range of 8 meters.

4. Solution and Metodology

4.1. Problem Formulation

The reconstruction system operates in an initially unstructured environment. The first critical phase that is executed is **Simultaneous Localization and Mapping (SLAM)**¹. During the SLAM process, the robots actively *explore* the workspace in order to generate a consistent map of the environment and, at the same time, estimate their own pose within such a map.

It is observed that the applicability of this architecture also extends to **outdoor** scenarios, in which the localization system can be integrated with, or replaced by, the use of the **Global Positioning System (GPS)**.

¹This preliminary phase is a prerequisite for the execution of the actual reconstruction system.

Subsequently, the system enters the **scanning and acquisition** phase. It is assumed that the system has **prior knowledge** of the location of the target object². The robots therefore move in a **coordinated** manner to strategically position themselves around the object.

Reach the correct position and the orientation towards the target objects, each robot follow an **Initial scanning**. for this scope, an RGB-D camera is used, from which not only the raw image but also the depth data relating to the object are extracted. After appropriate filtering and processing, each robot stores its own point cloud acquired with respect to a reference system that is the same for all robots.

This phase is **crucial**, since the application of the correct **roto-translation transformations** (which determine the robot pose with respect to the world frame) is essential to geometrically align the points acquired by the different robots within the same **global coordinate system**, thus avoiding the need to complicate the pipeline by introducing additional **Iterative Closest Point (ICP)** processes.[5].

The individual Point clouds are transmitted to a central **coordinator node**. The coordinator performs the merging of the datasets. In order to determine the regions of the object that still require observation, an algorithm based on **Gaussian distribution** and **clustering** is applied. The goal is to generate **patches** of currently unobserved regions with insufficient density or resolution and to identify the corresponding **centroids**, which represent the subsequent **optimal viewpoints**.

Once the new viewpoints have been identified, the robots perform a new **coordinated** motion, thus completing the **observation cycle** (scan-move-merge-plan). This iterative process continues until the object has been reconstructed with the desired level of accuracy and completeness, i.e., until a predefined observation threshold is exceeded or no further points to be acquired are detected.

4.2. Program Description

For proper operation, the project has been divided into packages in order to independently test the different phases of the system:

- **Vision system:** Package responsible for managing the robots' vision system (executed in parallel during runtime), which exploits both multi-threading and GPU acceleration to increase acquisition and computation speeds.
- **Navigation system:** Package in charge of navigation services and path planning for the robots.

²Although the approach can be extended to autonomous exploration and object discovery, for **design choice** and focus reasons, such an extension has been considered only at a theoretical and logical level in the current implementation.

- **Coordination system:** System responsible for point cloud merging and for the management of subsequent target points.
- **Main logic:** Core of the system in which the state machine and the communication among the different robots are managed.

4.3. Localization and Navigation System

For the localization component, the NAV2 library has been employed, which, as previously mentioned, allows the use of SLAM in the initial phase in order to obtain a first representation of the environment map. Subsequently, four services were developed that, within the state machine, enable the robot to be moved according to different motion modalities, depending on the current system state. The service are:

4.3.1. RandomPath

The *RandomPath* service automatically generates a random destination within a given distance from the robot, starting from its current position. The target point is determined by selecting a distance r within the interval $[r_{\min}, r_{\max}]$ and an angle θ in $[0, 2\pi)$; the target position is then computed as:

$$x_{\text{goal}} = x_{\text{robot}} + r \cos \theta$$

$$y_{\text{goal}} = y_{\text{robot}} + r \sin \theta$$

where $(x_{\text{robot}}, y_{\text{robot}})$ are the coordinates of the actual robot. The final orientation is set along the direction of the selected angle. Once the target point has been determined, the path planning system is triggered to compute the trajectory toward the destination.

4.3.2. SpecificPath

The *SpecificPath* service takes as input the desired target position to be reached. The only operation performed consists in assigning this position as the new navigation goal, thereby triggering the path planning process.

4.3.3. ArchPath

The *ArchPath* service enables the robot to follow a circular arc defined by a center (x_c, y_c) and a radius r , starting from the projection of the current position and ending at the projection of the target point onto the same circumference. Given the points $(x_{\text{robot}}, y_{\text{robot}})$ e $(x_{\text{goal}}, y_{\text{goal}})$, compute the projection:

$$(x_{\text{start}}, y_{\text{start}}) = \left(x_c + r \frac{x_{\text{robot}} - x_c}{d_{\text{robot}}}, y_c + r \frac{y_{\text{robot}} - y_c}{d_{\text{robot}}} \right)$$

$$(x_{\text{end}}, y_{\text{end}}) = \left(x_c + r \frac{x_{\text{goal}} - x_c}{d_{\text{goal}}}, y_c + r \frac{y_{\text{goal}} - y_c}{d_{\text{goal}}} \right)$$

where d_{robot} and d_{goal} denote the respective distances from the center. Between the two angles identified by

these points, a sequence of waypoints is generated and distributed along the arc.

$$x_i = x_c + r \cos \alpha_i$$

$$y_i = y_c + r \sin \alpha_i$$

For equally spaced values of α_i between the start and goal angles. The orientation at each waypoint is chosen to be tangent to the arc, in order to optimize the robot motion along the trajectory.

4.3.4. Rotation

The *Rotation* service allows the robot to orient itself toward a point of interest, typically the center of the object detected by the sensors. Given the target point (x_o, y_o) and the current robot pose $(x_{\text{robot}}, y_{\text{robot}}, \theta_{\text{robot}})$, the desired orientation angle is computed as:

$$\theta_{\text{des}} = \text{atan2}(y_o - y_{\text{robot}}, x_o - x_{\text{robot}})$$

Orientation error define by:

$$e_{\theta} = \theta_{\text{des}} - \theta_{\text{robot}}$$

To control the rotation, we use a proportional gain (based on PID control), where the angular velocity ω is compute like:

$$\omega = K_p e_{\theta}$$

where K_p is the gain of the proportional part.

The motion terminates when the angular error falls below a predefined threshold (approximately 5°), indicating that the robot is correctly oriented toward the object.

In addition to these services, two further services, namely *pause* and *stop*, have been implemented. These services are used to temporarily suspend or completely stop the robot motion and are integrated within the motion logic of the state machine.

4.4. Detection and Vision System

The perception system has been developed by integrating Machine Learning techniques with classical Computer Vision methods. In particular, the *YOLO* (You Only Look Once) architecture has been adopted for object recognition. Specifically, the neural network was fine-tuned on a customized dataset through a dedicated labeling phase, thereby ensuring a high detection confidence.

Once the robot is correctly positioned, the vision system manages the three-dimensional acquisition through three services executed sequentially:

- **Trigger Detection:** Object identification and 2D bounding box segmentation.
- **Point Cloud Extraction and Transformation:** Projection of the points into the 3D space and transformation into the global reference frame.
- **Filtering and Saving:** Point cloud cleaning and storage for the subsequent reconstruction stages.

4.4.1. Point Cloud Extraction and Transformation

The generation of the point cloud is performed by combining the RGB image and the depth map provided by the RGB-D camera. For each pixel (u, v) belonging to the bounding box detected by YOLO, the spatial coordinate $P_{cam} = [x_c, y_c, z_c]^T$ in the camera optical frame is computed using the *inverse perspective projection model* [8], which combines the 2D pixel location with the depth parameters of the camera:

$$z_c = d(u, v), \quad x_c = \frac{(u - c_x) \cdot z_c}{f_x}, \quad y_c = \frac{(v - c_y) \cdot z_c}{f_y} \quad (1)$$

where $d(u, v)$ denotes the sampled depth value, while (f_x, f_y) and (c_x, c_y) represent the focal lengths and the coordinates of the optical center, respectively (camera intrinsic parameters).

Subsequently, in order to make the data consistent with the environment map, the points are transformed from the camera reference frame (RF_{cam}) to the odometry or global reference frame (RF_{odom}) by exploiting the ROS TF2 system.

This process ensures that the extracted point cloud preserves its correct spatial position regardless of the subsequent motion of the robot.

4.4.2. Filtering and Saving Process

The raw point cloud extracted from the bounding box inevitably contains noise and points not belonging to the target object (background). To isolate the target, a filtering pipeline has been implemented, structured in the following stages:

1. **Floor Removal:** The lowest points of the point cloud are removed by retaining only those above a threshold z_{min} , thus eliminating the floor.
2. **Robot Proximity Filter:** Points located beyond a certain Euclidean distance from the robot are filtered out, thereby reducing distant background noise.
3. **Statistical Outlier Removal (SOR):** Isolated points are removed by analyzing the average distance from their k -nearest neighbors, thus eliminating sparse sensor noise.
4. **Voxel Down-sampling:** The point cloud is down-sampled using a voxel grid (voxel size 0.01 m) in order to uniform the point density and reduce the computational load.

At the end of the filtering process, the optimized point cloud, including the estimated and correctly oriented normals, is saved in .ply format.

4.5. Mesh Coordination Strategy

The coordination phase aims at determining the optimal viewpoints (*Next Best View* [6]) to complete the reconstruction of the object by analyzing the partial point

clouds acquired by the robots. To achieve this goal, two distinct approaches have been investigated: a probabilistic mapping based on Bayes' theorem and an observation density estimation based on a Gaussian distribution.

4.5.1. Initial Approach: Multi-Robot Bayesian Mapping

Initially, inspired by the state of the art in robotic reconstruction, a pipeline based on *Occupancy Grid Mapping* [2] was implemented. The algorithm subdivides the space into a three-dimensional voxel grid and exploits **raycasting** to simulate the visual ray from the robot toward each voxel.

The process is based on the recursive update of the occupancy probability of each cell m_i using the logarithmic form of Bayes' theorem, which allows the values to be summed at each iteration:

$$l(m_i | z_{1:t}) = l(m_i | z_{1:t-1}) + l(m_i | z_t) - l_0 \quad (2)$$

In other words, the current belief is equal to the previous belief $l(m_i | z_{1:t-1})$ plus the new sensor information $l(m_i | z_t)$, minus the prior uncertainty l_0 , where z_t denotes the observation at time t derived from the ray-casting process. The implemented script performed the following steps:

1. **Voxelization:** Creation of a uniform grid around the merged point cloud.
2. **Inverse Sensor Model via Ray-casting:** For each robot pose, virtual rays were cast. If a ray traversed a voxel without hitting any point, the corresponding variable was decremented ($L_{free} = -1.0$); if it intersected a point within the voxel, it was incremented ($L_{occ} = +1.0$).
3. **Centralized Consensus:** In a multi-robot scenario, the local maps were fused by computing their average in order to ensure consistency among the observations.

However, this approach proved to be computationally demanding and highly sensitive to noise, resulting in a large number of false positives.

4.5.2. Current Approach: Gaussian Observation

To overcome the computational limitations of volumetric reconstruction based on *raycasting*, a hybrid approach that combines statistical analysis and memory integration has been developed. The goal is to obtain a continuous measure of the *Observation Quality* of each region of space, thus abandoning the binary classification (Occupied/Free).

Gaussian Weighting and Voxelization. To overcome the limitations of raycasting, a lighter approach based on the **Gaussian distribution** has been developed, which replaces the binary Occupied/Free classification with a continuous measure of *observation quality*. For each point p_k , a weight w is defined according to its position within the statistical distribution of the local point cloud (estimated

through the covariance matrix), thus realizing a Gaussian decay that favors points closer to the centroid and aligned with the principal axes:

$$W(p_k) = \exp\left(-\frac{d_{\text{elliptic}}^2}{2\sigma^2}\right) \quad (3)$$

The value $W(p_k)$ becomes more normalized and memorized inside the array `point_t`.

Voxelized Aggregation: The space is discretized into a uniform grid of voxels (or *boxes*); for each cell C_j , the **Mean Observation** $\bar{O}_j$ is computed as the average of the weights w of the points contained within it.

Belief Reinforcement: In addition to the Gaussian distribution, a temporal integration with probabilistic reinforcement is introduced. In the subsequent scan, besides computing the mean of the weights of the newly acquired points within each voxel, a small fraction of the weights from the previous iteration is added, so as to avoid unnecessary re-observation of regions that have already been sufficiently confirmed. In other words, if an already observed cell is covered again by the current scan (`is_visited_again = True`), its historical value $\bar{O}_j^{(t-1)}$ is increased by an additive term δ_{obs} (set to 0.1):

$$\bar{O}_j^{(t)} = \min\left(\bar{O}_j^{(t-1)} + \delta_{obs}\right) \quad (4)$$

In this way, the confidence over regions confirmed by multiple observations is increased, while points from subsequent scans with the same spatial location are discarded in order to avoid duplicates.

Next-Best-View. For the automatic generation of exploration targets, the following procedure is adopted:

1. **Gap Identification:** Cells with an `avg_observation` value lower than a critical threshold (`observation_threshold`) are selected, thereby obtaining a set of clusters.
2. **Spatial Clustering:** The centroids of these clusters are grouped using the DBSCAN algorithm [4].
3. **Targeting:** The centroids of the resulting clusters identify the least explored regions and are projected onto a trajectory to guide the robots toward the parts of the object that require further sampling.

For further details, refer to the images of the cone scan in the *Results* section.

4.6. Main Logic

This component is used for the management of the robotic fleet and coordinating consensus. The implemented software architecture follows a hybrid paradigm that integrates concepts derived from *Behavior Trees* and *Finite State Machines*.

The operational flow is divided into sequential macro-blocks (States). The transition between consecutive states is regulated by a synchronization mechanism based on feedback signals, referred to as “**ticks**”. The central node waits for the completion tick from *all* agents before proceeding, ensuring that critical operations (such as image acquisition) are performed only when the entire system is in the correct configuration (e.g., robots stationary and properly positioned).

The *Main Controller* orchestrates the following responsibilities:

- **Reconstruction Cycle:** Supervises the iterative loop of navigation, perception, and map updating until a reconstruction quality threshold (`observation_mean`) or a maximum number of iterations is reached.
- **Multi-Robot Synchronization:** Manages asynchronous communications with the navigation and vision nodes, using *Condition Variables* to block the execution of the logic until the distributed tasks are completed.
- **Dynamic Goal Assignment:** At each iteration, receives a set of *Next Best Views* (NBVs) from the coordination system. A greedy optimization algorithm is employed to assign each target to the closest robot, minimizing the overall fleet displacement through a cost function based on the Euclidean distance:

$$(r^*, g^*) = \arg \min_{r \in R, g \in G} \|P_r - P_g\| \quad (5)$$

where P_r denotes the current pose of the robot and P_g denotes the goal pose.

4.6.1. Execution Pipeline

The state logic executed by the controller (`state_machine`) strictly follows the steps reported below at each iteration:

1. **Navigation Phase:** The robots are dispatched toward the computed observation points. In the initial phase, predefined checkpoints are used, while the subsequent phases rely on dynamically computed arc trajectories.
2. **Rotation Alignment:** Once the target positions are reached, a synchronous command is issued to rotate all robots toward the object.
3. **Vision and Acquisition:** The vision pipeline is activated, including YOLO detection triggering, point cloud extraction, and filtering. The system waits for the acknowledgment (*tick*) from each robot.
4. **Coordination Trigger:** The coordination service is invoked to merge the newly acquired data and compute the optimal poses for the next iteration.

5. Results

Also for the final results, the obtained outcomes were visually analyzed, together with several estimates with respect to the corresponding ground-truth values.

5.1. Qualitative Results

5.1.1. Centroid of the Central Body Point

From the analysis, it can be observed that, as the number of iterations increases, the re-computation of the centroid becomes progressively more accurate. This improvement is mainly due to the increasing number of points belonging to the object itself that are incorporated into the reconstruction.

5.1.2. Observations on Next Best View

Thanks to the clustering strategy and the Next Best View principle, at each iteration the system increases its overall mean observation value, rising from 0.5 to 0.69 in the final result. At first glance, this behavior can be attributed to the variance of the Gaussian distribution being set to a relatively high value, which assigns greater weight to the boundary points of each point cloud, as these are the least visible regions.

Another interesting aspect is that the Gaussian observation distribution model can also be applied locally to specific regions of the scan, thereby making the scanning process more complex but also more detailed. In other words, in the case of a camera with rotational capabilities, it would be possible to prioritize certain regions over others.

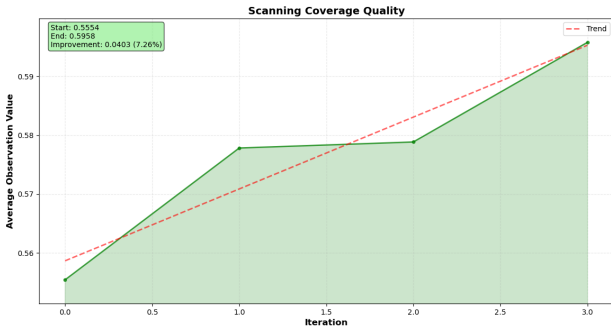


Figure 1: mean observation at each iteration

5.1.3. point cloud estimate compare with real object

Another relevant result is that the proposed algorithm also operates correctly with a single robot. When only one robot is employed, the selection of the subsequent view-points and the improvement of the object's mean observation value at each iteration become much more visually evident. In any case, the system has been designed to operate with an indefinite number of mobile robots.

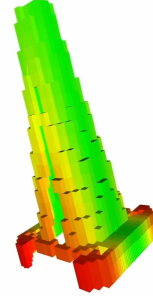


Figure 2: Voxelization of a cone during coordination phase

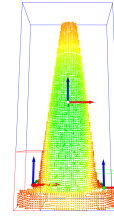


Figure 3: Clusters of points to observe with greater attention

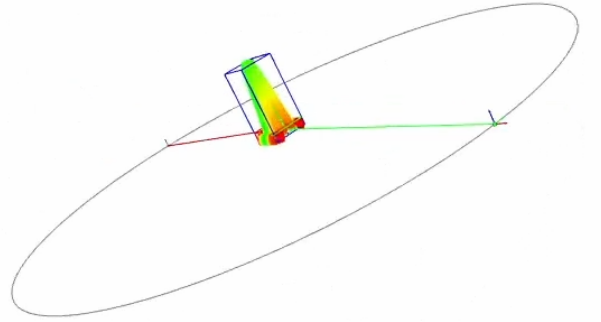


Figure 4: Next position for the robots

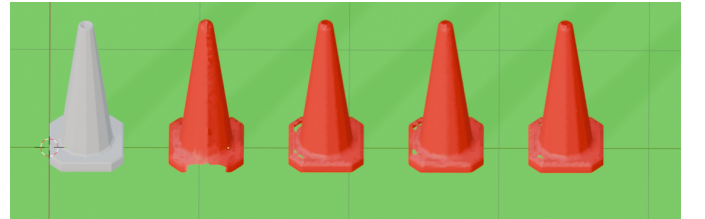


Figure 5: Comparison between the model found inside Gazebo(white) and the different iterations of the scanning algorithm (red)

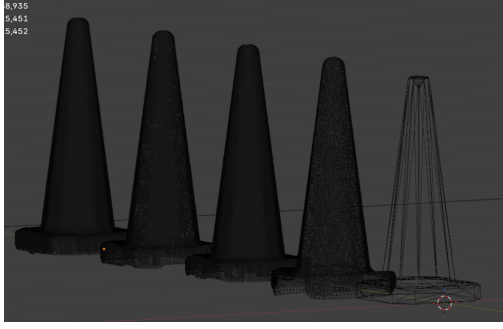


Figure 6: The wire-frame of the generated models, on the far right the original model composed of 166 triangles, on the far left, the final model with 119.536 triangles

5.2. Quantitative Results

To obtain quantitative results for the algorithm we needed a way to compare the ground truth 3D model with the reconstructed mesh removing the possible error added by the error in the estimated position of the object. To do obtain such values we needed to take in account multiple factors:

- The mesh obtained by the reconstructed pointcloud will have an order of magnitude more in number of vertices and triangles as shown in Fig. 6
- The estimated mesh could have holes.
- The generated pointcloud is not centered at the origin
- Even if we know the coordinates of the scanned object, we have to also take into account the noise in the frame of reference of the robots.
- The model imported from Gazebo has a different scale and different axis frame of reference

5.2.1. General mismatch of the model and the pointcloud

To solve the problems relative to the miss-match of the reference axis and scales we:

1. Translated the pointcloud to the origin of the axis
2. Rotated the reference mesh such to align the axis of the 2
3. Scaled the reference mesh to the dimension reported by Gazebo

5.2.2. The measurement of error

Since the used methods for determine the error we selected are heavily dependent on distances, we will further refine the alignment between the pcl and the mesh to remove any noise generated by the noise in the position in which the robot believe to be in. First of all we will generate a uniformly distributed pcl of 50000 points on the reference model. We used the RANSAC algorithm on a voxelized version of 2 pcls to roughly align the models. Than used the Iterative Closest Point algorithm to further improve the registration between the 2 pointcloud . Now that

the 2 pcls are aligned we generate the mesh from the estimated pointcloud using the Poisson reconstruction method. We will than place 50000 uniformly distributed points on the reconstructed mesh as shown in Fig. 7 to compute the Chamfer and Hausdorff distances between each iteration of the reconstruction algorithm and the reference mesh .

Chamfer Distance from original			
0.0210	0.0167	0.0166	0.0166
Hausdorff Distance from original			
0.1949	0.1946	0.1942	0.1941

Table 1: Distances/error between the original object and the generated mesh

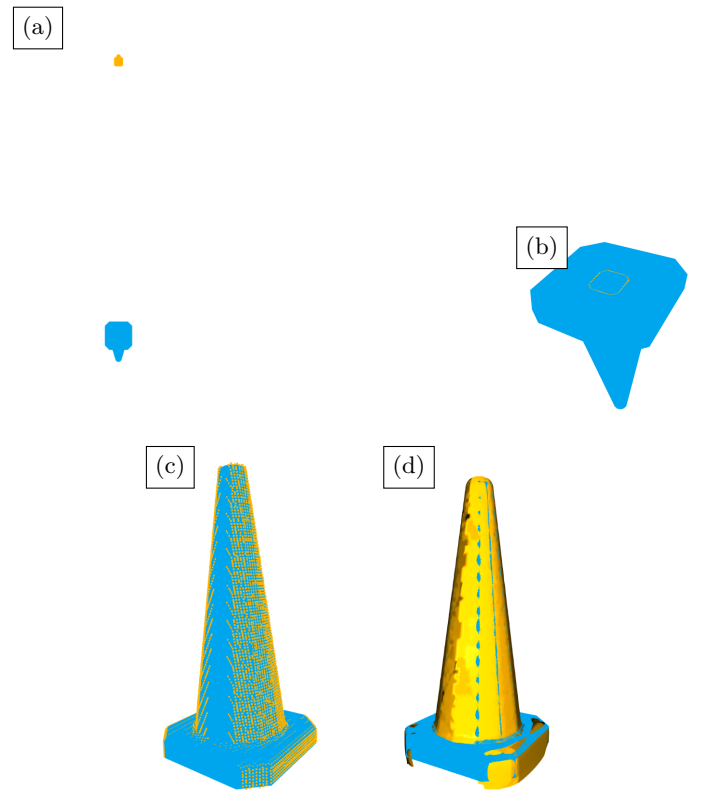


Figure 7: These 4 images are the output for the steps done in the comparing algorithm. We have: (a) the point cloud and the mesh as they are loaded in the script; (b) the two objects centered on the origin but with different scales; (c) the reference object rescaled and both meshes voxelized; (d) the final pointclouds from which the distances will be computed.

6. Conclusion

Several aspects related to collaborative robotics and multi-sensor data fusion have been acquired throughout the development of this project. Furthermore, a more developer-oriented approach has been addressed by focusing on resource optimization, such as the use of GPU acceleration and multi threading, which significantly contributed to improving the overall system performance.

The integration of multiple areas of expertise also enabled the development of a more solid and modular infrastructure. In particular, each node was designed with modularity in mind, thereby ensuring re-usability and facilitating its integration into future projects.

6.1. Problems encountered

The main issues encountered during this project were related to the management of large amounts of data generated by multiple robots, which made the system unstable and caused network congestion. Since the project was developed in a simulated environment, handling all the robots at the same time represented one of the first challenges we had to face. Initially, the system suffered from instability, packet loss, and sudden crashes. This led us to reflect on the convenience of using more centralized systems for complex processes such as point cloud analysis, as well as the adoption of multi threading paradigms for state machine management and the use of the GPU for the object detection tasks. All these components allowed us to achieve a smoother and better-managed system.

Another critical issue concerned multi-robot management. An initial idea was to use omnidirectional (holonomic) systems, which would have allowed us to avoid long and complex movements. However, due to strict time constraints, we opted for a simpler solution such as differential-drive robots. In any case, the algorithm has been designed to be adaptable to more complex systems such as drones and other types of robots. In such scenarios, it will be necessary to adjust filters and logical layers in order to optimize the system's performance.

6.2. Current limitations include:

- **Computational power:** currently, coordination is handled from a more centralized point of view, which limits scalability and performance.
- **Environmental complexity:** using more complex environments such as IsaacSim or similar platforms would make data management more realistic and noisier, thus allowing the analysis of more difficult-to-interpret data with a better interface to real-world conditions.
- **More accurate detection:** greater labeling and higher precision in supervised learning improve the detection algorithm in terms of object mapping and centering.
- **Use of additional next best view algorithms:** applying further next best view algorithms can help in finding points inside the object as well. Moreover, using a more dynamic camera movement in space can increase the scanning of the required object areas, avoiding unnecessary robot movements and improving overall efficiency.

- **Testing with large objects:** for project and time constraints, a simpler approach was chosen to verify the feasibility of the algorithm. However, if larger objects such as cars or buildings were to be scanned, it would be necessary to adjust filters, as they could otherwise introduce issues.
- **Web application integration:** to better visualize the point cloud interface, one possible improvement is the integration of a more advanced GUI that allows interaction with the data, such as reconstruction and other mechanisms, with the goal of providing a proper user interface.

References

- [1] Z. Li, Q. Zhang, Q. Sun, and Y. Huang, "A distributed framework for multiple UAV cooperative target search under dynamic environment," *Journal of the Franklin Institute*, vol. 361, no. 8, p. 106810, 2024.
- [2] K. M. Wurm, A. Hornung, M. Bennewitz, C. Stachniss, and W. Burgard, "OctoMap: A Probabilistic, Flexible, and Compact 3D Map Representation for Robotic Systems," *ICRA Workshop on Best Practice in 3D Perception and Modeling for Mobile Manipulation*, 2010.
- [3] S. Thrun, "Learning Occupancy Grids With Forward Sensor Models," *Autonomous Robots*, vol. 15, pp. 111-127, 2003.
- [4] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise," *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96)*, pp. 226-231, 1996.
- [5] P. J. Besl and N. D. McKay, "A Method for Registration of 3-D Shapes," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 14, no. 2, pp. 239-256, 1992.
- [6] S. Kriegel, C. Rink, T. Bodenmüller, and M. Suppa, "A Surface-Based Next-Best-View Approach for Automated 3D Model Completion of Unknown Objects," *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 4869-4874, 2011.
- [7] Z. Jia, Y. Li, Q. Hao, and S. Zhang, "PB-NBV: Efficient Projection-Based Next-Best-View Planning Framework for Autonomous 3D Reconstruction," *arXiv preprint arXiv:2501.10663*, 2025.
- [8] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.

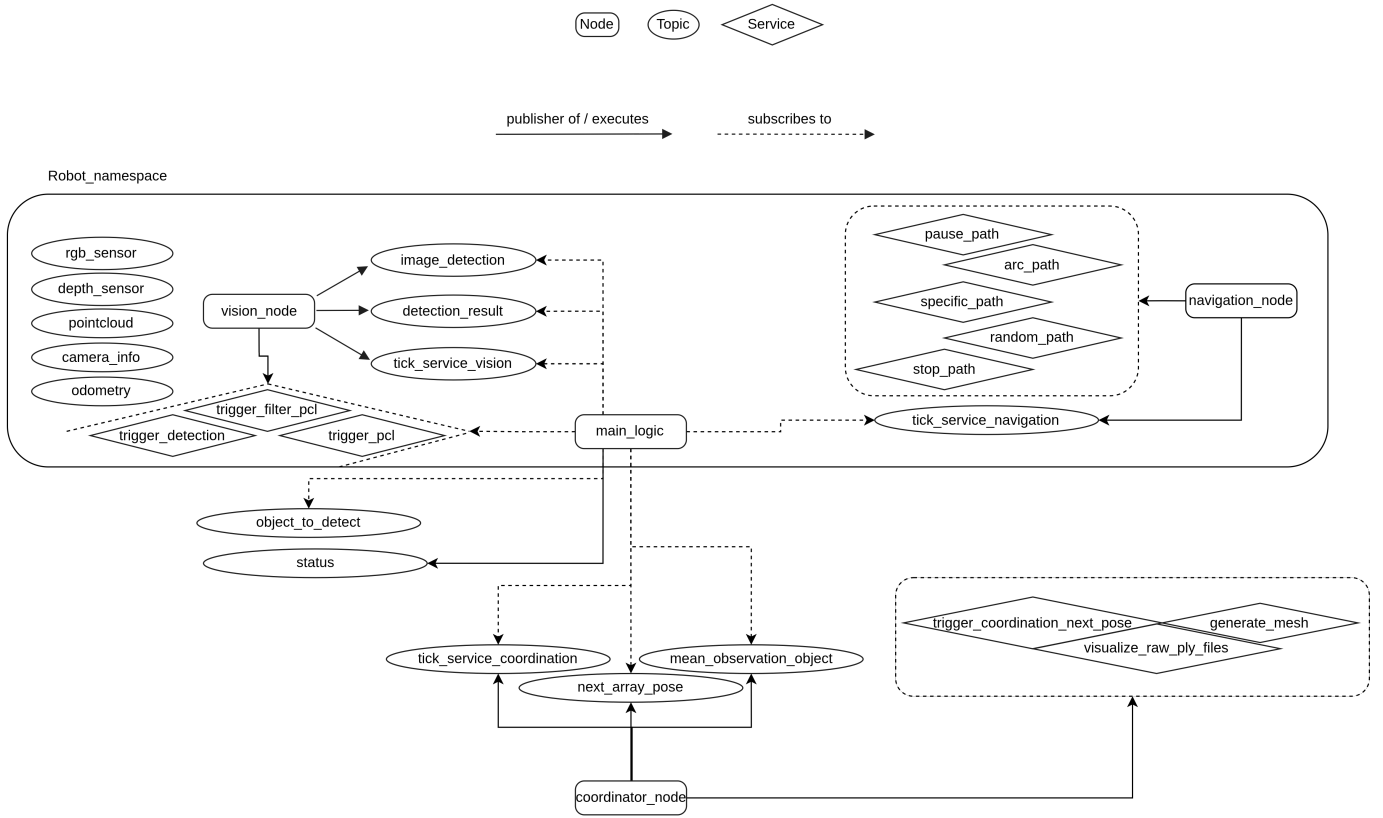


Figure 8: This is the map representing the topics and services within our project. The nodes within the robot_namespace box are the nodes replicated for each robot

[9] D. Gutierrez-Gomez, W. Mayol-Cuevas, and J. J. Guerrero, "Dense RGB-D Visual Odometry Using Inverse Depth," *Robotics and Autonomous Systems*, vol. 75, pp. 571-583, 2016.